

Worksheet No. - 01

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 2nd

Subject Name: Artificial Intelligence Tools

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 16.01.2026

Subject Code: 25CAP-657

Aim/Overview of the practical: To implement the Breadth-First Search (BFS) algorithm for traversing a graph, starting from a given source node, using a queue data structure, and to understand how BFS explores all nodes level by level while ensuring the shortest path is found in an unweighted graph.

Objective: The objective of this experiment is to understand and implement the Breadth-First Search (BFS) algorithm for traversing a graph. It aims to study how BFS uses a queue to explore nodes level by level starting from a given source node, ensuring that the shortest path is found in an unweighted graph. This experiment also helps in understanding the practical applications of BFS in areas such as network traversal, web crawling, and social networking systems.

Procedure/Algorithm/Code

Algorithm:

1. Start the program.
2. Import the required libraries (such as collections.deque for queue operations).
3. Represent the graph using an appropriate data structure
4. Define a BFS function that takes the graph and starting node as input.
5. Initialize an empty set (or list) to store visited nodes.
6. Create a queue and insert the starting node into the queue.
7. Mark the starting node as visited.
8. While the queue is not empty:
 - Remove the front element from the queue.
 - Display or process the current node.
 - Traverse all adjacent nodes of the current node.

- If an adjacent node is not visited, mark it as visited and add it to the queue.
9. Continue the process until all reachable nodes are visited.
 10. Stop the program.

Code:

```
import matplotlib.pyplot as plt
import networkx as nx
from collections import deque
```

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': ['I']
}
```

```
# Create directed graph
```

```
G = nx.DiGraph()
```

```
for node in graph:
```

```
    for neighbor in graph[node]:
```

```
        G.add_edge(node, neighbor)
```

```
# Draw graph
```

```
plt.figure(figsize=(9, 4))
```

```
pos = nx.spring_layout(G)

nx.draw(
    G, pos,
    with_labels=True,
    node_color='skyblue',
    node_size=1500,
    edge_color='gray',
    font_size=12,
    font_weight='bold'
)

plt.title("Graph Visualization for BFS")

plt.show()

# BFS function
def bfs(graph, start_node):
    visited = set()
    queue = deque()
    bfs_order = []

    visited.add(start_node)
    queue.append(start_node)

    print("BFS Traversal Order:")

    while queue:
        current_node = queue.popleft()
```

```
print(current_node, end=' ')\n\nbfs_order.append(current_node)\n\nfor neighbor in graph.get(current_node, []):\n    if neighbor not in visited:\n        visited.add(neighbor)\n        queue.append(neighbor)\n\nreturn bfs_order
```

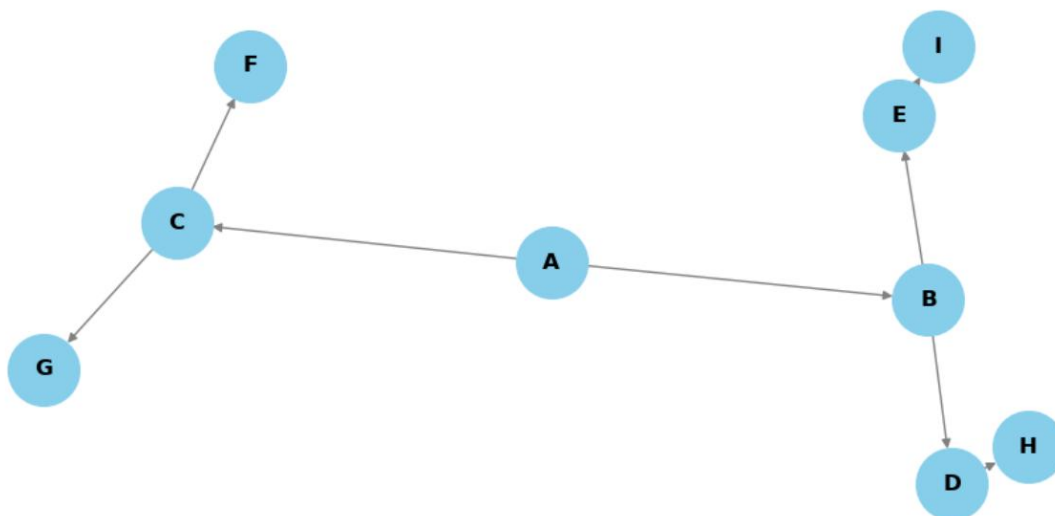
Run BFS

```
start_node = 'A'
```

```
order = bfs(graph, start_node)
```

Output

Graph Visualization for BFS



BFS Traversal Order:
A B C D E F G H I

Learning outcomes (What I have learnt):

1. Understood the concept and working of the Breadth-First Search (BFS) algorithm.
2. Learnt how to represent graphs using an adjacency list in Python.
3. Gained practical experience in using a queue (FIFO) data structure.